

Action modding

This page is about modding actions available in game. For the game effect hooks, see [On actions](#)

Action modding is a powerful way of creating new interactions between countries and other objects in the game world. Many different types of actions in the game share much common syntax, specifically the interaction target system, allowing designers to give end-users and AIs alike easy and efficient ways of performing actions.

Please help with verifying or updating older sections of this article.
At least some were last verified for [version](#) 1.0.

İçindekiler

Common syntax

- [Potential, allow and effect](#)
- [Interaction targets](#)
 - [Visibility](#)
 - [Allow null and allow self](#)
 - [Localization](#)
 - [Optimization - sourcing](#)
 - [Optimization - caching](#)
 - [Optimization - other](#)
 - [Looking for a value](#)
 - [Map Modes](#)
 - [UI & Sorting](#)
 - [Columns in detail](#)
 - [Column definition](#)
 - [Technical details](#)
 - [Syntax](#)

- [Action AI](#)
- [Automation AI](#)
- [Price](#)
- [Cooldown](#)
- [Sound](#)
- [Message settings](#)
- [Exclusive group](#)

Generic actions

- [Generic action type](#)
- [Other aspects](#)
- [Generic action lists](#)
- [Using generic actions in GUI](#)
- [Localisation](#)
- [Messages](#)

Character interactions

- [Specification](#)

interaction type and categoriesTarget countryAcceptance and rejectionDiplomat usage and costsDiplomatic costsLocalisation**Cabinet actions**Cabinet action basicsPotential and allowCabinet effects and modifiersCabinet AIAction progress and trackingLocalisation**Other**ResolutionsMissionsPeace treaties**References**

Common syntax

Common action syntax includes basic aspects like triggers, effects, AI utilities. Most important are the interaction targets which allow for great versatility.

Potential, allow and effect

Like many other objects, actions also have a field for `potential`, `allow` and `effect`. For the former two, `scope:actor` is provided for the country, and `effect` will receive `scope:actor` as well as all scopes from interaction targets.

Interaction targets

Central to the action syntax is the interaction targets, which can be recognized by the `select_trigger`. The central argument in a interaction target is the `looking_for` entry, which determines the scope the select trigger is "looking for". For instance, a `looking_for = country` will bring up a basic list of countries to select from and will allow to select the country on the map. Lesser known scopes like `looking_for = value` are also supported. After an object is selected, it is set for future use as a scope outlined in `target_flag`. For example:

```
select_trigger = {
    looking_for = country
    target_flag = target_country
}
```

A country selected in this block can be later referred to using `scope:target_country`.

Visibility

If it is desirable for the `select_trigger` to only be usable conditionally, its visibility can be limited using `show_if` trigger. One can also control whether certain targets in the list are visible/selectable using `visible` and `enabled` triggers respectively. There is also `selected` trigger, which seems to check if a selected target is "currently selected". Can be referred to using `SelectInteractionTargetGlue.IsSelected`. If no targets are visible/enabled, hovering over the action will display the text provided in `none_available_msg_key`. Moreover, the game may allow the end user to see why targets are unavailable by showing the triggers from `enabled`. This can be disabled by using `show_why_not_enabled = no`, or further expanded with triggers from `visible` by using `show_why_not_visible = yes`.

```
select_trigger = {
    looking_for_a = province
    source = actor
    target_flag = target
    name = "choose_province"
    none_available_msg_key = "no_provinces_available"
    visible = {
```

```
# Do not show if there are no locations in the province that
```

```

    }
  }
  show_if = {
    scope:actor.capital instead of scope:target # Only allow for selection if actor has more than one province. Otherwise we will use
    scope:actor = {
      num_provinces > 1
    }
  }
  enabled = {
    population >= 10 # Allow to select only the provinces that are populated enough
  }
  show_why_not_visible = yes # If no province is visible, also show the visible requirements in the tooltip next to
  none_available_msg_key
  show_why_not_enabled = yes # Default behavior.
}

```

Allow null and allow self

If `allow_null = yes` is set, an option will be displayed to select "None". Selecting it will push through the "select_trigger" but will not set the flag outlined in `target_flag`. `allow_self = yes` can be used to include the actor country in the interaction target that looks for a country. The default behavior is that actor country is not included.

Localization

`name key` is used to set a localization key that will be used to localize the top text in the selection window. `none_available_msg_key` key is used to set the text that will be displayed if there is no such target available.

Optimization - sourcing

Interaction targets can be optimized in a multitude of ways. The most basic optimization is the usage of `source`, which, when set, will try to source the scopes from the "source" object. The right side of `source` requires the key for an existing scope. If `scope:actor` exists, then it can be used in the following way:

```
source = actor
```

This behavior of sourcing depends on the scope provided and the scope the interaction target is looking for. The behavior of sourcing can then be further modified by the usage of `source_flags`, which allow further specialization and optimizations.

For scripting AI behaviors, `source_ai_override` and `source_flags_ai_override` exist, and work in much the same way except they override the behavior only for AI calculations.

Another way to source objects for the interaction target is to use `source_global_list` which allows providing a key of a global variable list from which the possible targets will be taken. In order to build such a target list on the fly, `interaction_source_list` can be used, which will source the target from a local list called `source`:

```

interaction_source_list = {
  formable_country:ITA_f = {
    add_to_list = source
  }
  formable_country:LOT_f = {
    add_to_list = source
  }
}

```

There is also a version of the above for the AI under `ai_interaction_source_list`.

List of all possible entries for `source` and `source_list`, based on the scope:

[Genişlet]

Optimization - caching

Interaction targets offer ways of optimization via caching. One of such ways is to use `cache_targets = yes` field, which will tell the AI calculations that the list of targets in this specific `select_trigger` does not depend on any prior interaction targets. If there is a guarantee that the current interaction target will not depend on a previous time, this field



Additionally, if the list of targets provided by an `interaction_source_list` is static, `cache_interaction_source_list = yes` should be used so as to ensure that the calculation for the source list is not done an unnecessary amount of times.

Optimization - other

You can provide `max_targets_for_ui` with an integer amount of targets that the UI should be delivered at a time.

`pre_evaluation_sort_value` is a script value which will determine the sorting order for AI, it is to be used with `pre_evaluation_number_to_evaluate_fully`. AI will first sort entries according to the script value and only evaluate the top entries according to the value set with the `pre_evaluation_number_to_evaluate_fully`.

Looking for a value

As numbers work slightly differently than other scopes, `looking_for_a = value` has special customization options. Both `max` and `min` script values can be used to limit the maximum and minimum value respectively, while `default` script value is used to set the default value of the interaction target. As value selection is a slider, the designer can also set a value for how much each discrete "step" should represent via `step` script value. The AI will evaluate different value combinations and this can be optimized using `ai_override_value`, which will make AI only evaluate that one script value.

Map Modes

Interaction targets also allow for much customization in terms of mapmodes. `map_mode` can be supplied with a map mode key to override currently viewed mapmode.

```
map_mode = raw_material
```

will show raw materials on the map as if the `raw_material` mapmode was being used.

One can also create a custom, inline mapmode using `map_color` and `secondary_map_color` in much the same fashion as in `scriptable mapmodes`. Tooltip text can also be added to locations with a localization key provided in `tooltip_msg_key`. If one wants only selectable locations to be colored in, `only_color_selectable = no` should be used, otherwise, all locations in the world will be colored by this mapmode, regardless of whether they can be selected in the interaction target or not.

UI & Sorting

The most important part of an interaction target UI is the list of targets, its display and its sorting. Those can also be customized extensively. A new column for sorting can be inserted into a `select_trigger` with the `column` property:

```
select_trigger = {
  looking_for_a = country
  column = {
    data = name
  }
  column = {
    data = population
  }
}
```

The keys provided on the right side of `data` are column definitions. The default sorting key will be the first column provided unless `default_sort` has been specified. `default_sort` will accept the key of the column that should be the default sort column.

There needs to be at least one column, otherwise the game will crash!

In terms of GUI customization, an interaction target can accept `top_widget` and `bottom_widget` which give the designer extra control over how the `select_trigger` panel looks. `top_widget` is meant to be given a GUI widget type that will be placed between the panel header and the search bar. For example:

```
top_widget = marry_noble_character_top
```



Columns in detail

The syntax for columns allows for creation of an inline column or a reference to an existing one that is contained within a Column definition.

A column definition can be inserted by using `data = <key>` as with a previous example, and the following widgets are used for defining a new one:

`widget entry` accepts a GUI type that will be placed as a target entry. The widget will have `InteractionTarget` datatype available.

`width` is used to set the width of the field at the top and below. `is_constant_width = no` is used to prevent those fields from taking up remaining space, which is the default behavior.

`fixed_height` can be used to tell the game about the height of the widget, if it is fixed. This allows for extra optimization.

`contains_select_target_button` is used to tell the game whether the widget provided has a `select_target_button` widget. If not, the game will create one. `single_widget_for_row` is used when the widget that is being used is intended to be used is meant to be the only one. The game will then error if more columns are added.

The main other component besides `widget` is `sort` which is used to determine the ordering. A single column can have multiple sort fields even though it is technically one column. Each sort needs a localizable key representing it with `sort_key`. A tooltip can be added to the sort header by using `sort_by_tooltip_key`. The default load order depends on what the sort is sorting by, being sorted ascendingly unless it is being sorted by text, in which case descending sorting is being used. This behavior can be overridden by using `sort_order = "Ascending"` and `sort_order = "Descending"`.

Moreover, two extra fields exist and are mainly used in sorting fields according to the cost of the target.

`sort_by_cost_button` accepts `left` and `right` in order to decide whether to sort under the cost which is under left or right click and `sort_by_cost_type` is used to determine whether that cost is under `click` or under `clickandconfirmorhold`.

The two main ways of sorting though are `sort_text` and `sort_value` which are used to sort the entries by their lexicographic and value order respectively. `sort_value` utilizes a script value calculated on the interaction target:

```

sort = {
    sort_value = {
        value = population
    }
    sort_by_tooltip_key = "LOCATION_SORT_BY_POPULATION"
}

```

While `sort_text` utilizes a script string:

```

sort = {
    sort_text = {
        if = {
            limit = {
                exists = root
            }
            value = "SORT_TEXT_LOCATION_OWNER_NAME"
        }
        else = {
            value = "MODIFIER_NONE"
        }
    }
    sort_by_tooltip_key = "LOCATION_SORT_BY_OWNER"
}

```

Column definition

Column definitions can be used as keys in `data`. They can be defined externally for reuse later, they outside of the definition space, they follow the same syntax.

Technical details

Column definitions are stored in `common/attribute_columns` folder, for example:

`/Europa Universalis V/game/in_game/common/attribute_columns/00_defaults.txt`

Syntax

first opening entry in the file is the type for which we are adding columns for:



The entries inside then represent the different attribute columns:

```
country = {
  name = {
    ...
  }
  owner_flag = {
    ...
  }
}
```

Action AI

Actions in Europa Universalis V have versatile built-in methods for handling AI. Most important is the `ai_will_do` script value, which determines if the AI is willing to do the interaction and by what factor. The field is provided with all interaction target scopes and `scope:actor`.

AI will evaluate actions according to their AI tick data. The `ai_tick` is used to determine whether the AI evaluates the actions daily, monthly or just does never evaluates it. Once the type of tick is decided, `ai_tick_frequency` script value is used to decide the interval between the checks.

If one wants to place additional requirements on AI, `ai_prerequisite` trigger can be used.

```
test_action = {
  ...
  ai_prerequisite = {
    current_year >= 1400    # Year is after 1400
    gold >= 200            # Country has at least 200 gold
  }
  ai_tick = monthly
  ai_tick_frequency = {
    value = 12             # evaluate this every 12 months
    if = {
      limit = {
        current_year >= 1500
      }
      value = 1            # if the year is at least 1500, evaluate it every month
    }
  }
  ...
}
```

Automation AI

Actions can be automated by player automation. This automation will use AI logic to determine optimal choices, but will use its own fields for automation, `automation_tick_frequency` and `automation_tick`, working analogically. `player_automated_category` is used to determine which automation category this action belongs to. Those are the following automated system keys:

- finances
- research
- trade
- productionmethods
- laws
- cabinet
- governmentreforms
- parliament
- estates
- exploration
- colonies
- cultureacceptance
- religiousdoctrines
- buildings
- closebuildings
- subsidizebuildings
- destroybuildings



- armybuilder
- navybuilder
- diplomacy
- rivals
- replacegenerals
- replaceadmirals

Price

The cost of actions is made convenient with the use of `price`, which accepts a scope that returns price object (most commonly the price data scope). Price uses price objects which define a cost. `price_modifier` is a scripted value that can be used to modify the effects of the price.

```
test_action = {
  ...
  price = price:control_the_food_market      # Price from common/prices folder
  price_modifier = {
    value = 1                                # The base price is 100%
    add = scope:actor.country_rank_level    # Which is further increased with the country rank of the actor
  }
  ...
}
```

By default, `scope:actor` (the country) pays the price, but this can be overridden using `payer`, which takes a scope on the right, for example:

```
payer = scope:recipient
```

By default, the price paid goes nowhere, but this can be changed to another target using `payee`:

```
payee = scope:target_country
```

If the designer wants the requirement part of the price but not actually enforce payment, they can use `should_execute_price = no`. This can be useful if the price is handled in a different way.

Cooldown

The `cooldown` field is used to add a cooldown to the action. The syntax of `cooldown` accepts a type which is a string identifying the cooldown. If the string is made to be common between actions, it will be shared cooldown. The duration of the cooldown is set by using scripted values under the following:

- years
- months
- days

Sound

You can assign a sound effect to the action with `sound`.

Message settings

By default, actions spawn messages according to their keys. `show_message = no` can be used to prevent messages from showing up. `show_message_to_target = no` can be used to prevent messages from showing up to the target of the action.

Further behavior of messages depends on the type of action that utilizes the interaction target syntax.

Exclusive group





Generic actions

Generic Action is the most basic implementation of an action in Europa Universalis V, but it does have a few extra additions over the common syntax.

Generic action type

The most important aspect of a generic action is its type, which heavily determines its behavior.

The following are the supported types:

Generic action

Game folder: common/generic_actions

Recommended top folder: in_game

File encoding: UTF-8 BOM

Injectable: ?

Replaceable: ?

Action type	Requires recipient	Expected recipient scope	Other Information
owncountry	Forbidden	Does not apply	Used for most basic interactions (default value)
parliament	Forbidden	Does not apply	Fetches with GovernmentView.GetParliamentActions GUI function.
religious	Required	religion	Fetches with CountryReligionLateralView.GetReligiousActions GUI function. Returns datamodel with all actions whose scope:recipient is the religion
religiousfaction	Required	religion	Fetches with ReligiousFactionGlue.GetActions GUI function. Returns datamodel with all actions whose scope:recipient is the religion
location	Required	location	Unused
internationalorganization	Required	international_organization	Fetches with InternationalOrganizationsView.GetActions GUI function. Returns datamodel with all actions whose scope:recipient is the international_organization
internationalorganizationparliament	Required	international_organization	Fetches with InternationalOrganizationsView.GetParliamentActions GUI function. Returns datamodel with all actions whose scope:recipient is the international_organization only if the IO has parliament.
situation	Required	situation	Fetches with SituationView.GetActionGroups GUI function. Returns action groups with all actions whose scope:recipient is the situation
disaster	Required	disaster	Fetches with DisasterView.GetActions GUI function. Returns datamodel with all actions whose scope:recipient is the disaster

Other aspects

If an action allows interacting with locations, one can use allow_multiple_targets for AI to target multiple locations at once. If one designs an action to designate a location as belonging to an arbitrary list, this can be used to make AI select multiple ones in batch.

Moreover, a message type can be assigned to a generic action with message = <message_type_key>.

The "match type and get added to a generic action list" behavior that stems from action type can be turned off using show_in_gui_list = no.

Generic action lists

There is a special database object that is used to optimize generic actions for AI. Those generic action lists are stored in common/generic_action_ai_lists, e.g: in_game/common/generic_action_ai_lists/black_death_list.txt



will evaluate the potential before evaluating the actions inside the actions. An example:


```

actions = {
    hide_from_black_death
    stop_hide_from_black_death
    isolate_cities_black_death
    stop_isolate_cities_black_death
    control_the_food_market
    stop_control_the_food_market
    close_the_borders
    stop_close_the_borders
    procure_remedies
    stop_procure_remedies
    segregate_the_infected
    stop_segregate_the_infected
    strict_quarantines
    stop_strict_quarantines
    sponsor_sin_forgiveness
    stop_sponsor_sin_forgiveness
    blame_the_minorities
    stop_blame_the_minorities
}

```

Using generic actions in GUI

The strongest aspects of generic actions is their use in GUI. By default, AI is able to use generic actions via the aspects provided but a user also needs to have an interface where they can interact with the actions. Some types deal with that in their own ways, but for some you will have to insert actions into GUI. There are multiple ways to do that.

Widgets that inherit from `action_button` hardcoded widget type can use `left_action` and other related gui attributes:

```

action_button_default = {
    size = { 115 30 }
    title = "OFFERMILACCTITLE"
    description = "OFFERMILACDESC"
    actor = "[ForeignCountryView.GetPlayer]"
    recipient = "[ForeignCountryView.GetCountry]"
    left_action = {
        action_name = "invite_to_international_organization"
        action_direction = "offer" (request, offer, cancellation, break)
        parameter = {
            parameter_name = international_organization_type
            parameter_value = "[GetInternationalOrganizationType('defensive_league')]"
        }
    }
    left_click_and_hold_action = {
        action_name = "scripted_relation"
        parameter = {
            parameter_name = scripted_relation_type
            parameter_value = "[GetScriptedRelationType('alliance')]"
        }
        action_direction = "cancellation"
    }
    right_action = {
        action_name = "subject_diplomacy_action"
        action_direction = "break"
        parameter = {
            parameter_name = target
            parameter_value = "[ForeignCountryView.GetCountry.GetDiplomacy.GetSubjectType]"
        }
    }
    right_click_and_hold_action = { action_name = "create_market" }
}

```

Additionally, the global function `PerformGenericAction( Arg0 )` can also be used to use actions.

Localisation

A generic action needs to have its title and description localized. The two strings to localize are `<key>` and `<key>_desc`.

Messages

A generic action needs to either have a generic message type under `PERFORM_<key>_ACTION` or a `WE_PERFORM_<key>_ACTION`. One may also provide `OTHER_PERFORMS_<key>_ACTION`.

A generic action needs to have a message type for it defined.

Character interactions

Specification

Besides the common syntax, `character_interactions` also have a few extra parameters:

`on_other_nation = yes` field is required to make an action be usable on characters that are not owned by the country (scope:actor).

`on_own_nation = yes` field is required to make an action be usable on characters that are owned by the country (scope:actor).

`is_consort_action = yes` field is required to make an action be usable on consort of the country.

`message = no` field can be used to disable a message being shown about an action being used on a character.

`context_menu_click_mode = click` is used to disable a confirmation dialogue for the interaction. The default is `clickandconfirmorhold`.

Game folder: `common/character_interactions`

Recommended top folder: `in_game`

File encoding: UTF-8 BOM

Injectable: ?

Replaceable: ?

In order to make a character interaction work, one needs to have a interaction target on a character representing scope:recipient that is looking for a character - that character will be the target of the interaction.

The array of all character actions is output with `CharacterLateralview.GetActions`.

Localisation

Character interactions require several strings to localize:

- `<key>` representing the title.
- `<key>_desc` for tooltip.
- `<key>_desc_specific` for message that is shown when using the action.
- `<key>_act` for action name.
- `<key>_act_past` for message that is shown when using the action.
- `<key>_past` for title when the action has been already performed, used for message that is created.

Country interactions

Country interactions are actions that are executed by a country on another country. In practice, they boil down to diplomatic actions and subject actions.

Interaction type and categories

All country interactions should have a type that determines if they are "diplomatic" or to be used on subjects. The two accepted keys are:

- `diplomacy`
- `subject`

If no type is provided, it will default to `diplomacy`.

Subject interactions will be automatically assigned to the subject actions category and will not require acceptance. Moreover, subject interactions will therefore be more optimized, as only subjects/overlord relationships will be evaluated.

Otherwise, a category needs to be assigned to an action using `category`. The categories can be added at will and must be localized then by localizing the key. All actions of the same category will be coupled together and might share their space with hardcoded diplomatic actions. Icons for new categories must be added in GUI itself in `diplomatic_actions_content` type.

Country interaction

Game folder: `common/country_interactions`

Recommended top folder: `in_game`

File encoding: UTF-8 BOM

Injectable: ?

Replaceable: ?

Target country





In practice this means that much of the diplomatic action effects will be executed with `scope:actor` as the caller, `scope:recipient` as the target country, and other parameters if applicable.

Acceptance and rejection

Country interactions can be made to require acceptance from the target country. This behavior is governed by the presence of `accept` script value and/or the presence of `diplo_chance`, both of which determine the acceptance reasons in their own specific ways. The lack of presence of those two keys will mean the interaction does not require acceptance.

- `diplo_chance` is a list of hard-coded fields that assign each different value its own weight in the calculation. More can be learned about this in AI Modding.
- `accept` is a script value that is delivered with the targetted action scopes.

If target country has a chance to respond but will refuse, it will fire the `reject_effect` with the parameters.

Ability to use the action can be further restricted by `block_when_at_war = yes`, which will block the action if the actor and recipient are at war with each other and the action requires acceptance.

`ai_limit_per_check` can be used to limit the maximum amount of countries AI will use this on per month.

Diplomat usage and costs

New country interactions will, by default, require a diplomat. This behavior can be overridden by using `use_enroute = no`. Requiring an enroute will mean that the action will use the same, global, 1 month action cooldown and will require 1 diplomat, which will be consumed when used.

`diplomatic_cost` is used, meanwhile, to add an additional diplomatic cost to the action.

Diplomatic costs

Diplomatic costs are containers used to define special costs for new diplomatic actions.

They are stored in `common/diplomatic_costs`.

Each entry in the file represents a diplomatic cost that can either hold a value for `spy_network` or `favors`.

```
new_test_cost = {
    spy_network = 30
    favors = 5
}
```

Localisation

Country interactions can require many different aspects to localize as they are treated as diplomatic actions.

- `<key>` must be localized to represent a title.
- `<key>_desc` must be localized to represent a description. This description is shown in the bottom textbox when hovering over the action in the diplomatic action view.
- `<key>_act` must be localized to represent what the action will do. This is shown above the textbox above description when hovering over the action where it says "This will have the following effect:"
- `PROPOSE_<key>` must be localized to represent the text in the confirmation dialogue. "Do we wish to do X? It will have the following effects:"
- `INCOMING_OFFER_<key>` must be localized to represent the text that the target country gets when an offer that can be accepted/denied gets used on them.
- `<key>_reject_effect_text` must be localized if the action can be denied/accepted
- `<key>_reject_effect_text_past` must be localized if the action can be denied/accepted
- `<key>_effect_text` must be localized
- `<key>_effect_text_past` must be localized



Cabinet actions

Cabinet actions are special actions that only utilize the interaction target syntax, but not the rest of the common syntax. They have their own mechanisms for triggers, applications and AI, but they still maintain the use of the `select_trigger` for the versatility in selecting targets.

Cabinet action basics

A primary requirement of a cabinet action is the selection of its `type = <type>`, which determines which category of ruler and cabinet member skill affects its efficiency. The three accepted values are:

- `adm`
- `dip`
- `mil`

`icon` is used to override the icon shown for the cabinet action. By default, the game will locate the icon by using `CABINET_ACTION_ICON_PATH` define (Vanilla value: `"gfx/interface/icons/cabinet_actions"`) and the action key. For example:

```
custom_action = {
    ...
    icon = pop_promote_action
    ...
}
```

By default, each action can only be used simultaneously by only cabinet slot. `allow_multiple = yes` can be used to prevent this behavior.

Potential and allow

As with many other objects, `potential` is a trigger on the country to determine if the cabinet action should be available to the country. Similarly, `allow` is also a trigger to determine the same, but if `allow` is not fulfilled, the action will not be hidden.

Cabinet effects and modifiers

Cabinet actions contain fields for three different type of modifiers.

- `country_modifier` gets applied to the country.
- `location_modifier` get applied to each location that is geographically part of the last Interaction Target. If the last `select_trigger` points to an area, every location in that area will be affected.
- `province_modifier` get applied to each province that is geographically part of the last Interaction Target, with logic much similar to `location_modifier`.

Societal values may also fire a `monthly_effect`. The `ROOT` in that effect is the country that is using the cabinet. Various scopes from the Interaction Targets are available. Besides being able to fire effects on `monthly`, cabinet actions may also fire effect when they are first set, and when they are deactivated with `on_activate` and `on_deactivate` respectively. When the cabinet action finishes implementing, it will also fire `on_fully_activated` effect, with the parameters.

Additionally, `societal_values` exists and it accepts a floating point number. The existence of this will make the action behave like `change_societal_values` vanilla action, allowing the user to select a certain societal value to push towards according to the value provided.

Cabinet AI

One can use `ai_will_do` which, as a script value, will take the Interaction Targets scopes and its result will be treated in addition to default AI logic.

Cabinet action

Game folder: `common/cabinet_actions`

Recommended top folder: `in_game`

File encoding: `UTF-8 BOM`

Injectable: ?

Replaceable: ?



Action progress and tracking

Cabinet actions can be given an "implementation duration". This is done via providing days, weeks, months or years scripted value.

When a cabinet action is utilized on certain geographical area, a map marker can be made to appear. This map marker can be given additional settings by using `map_marker`, which contains additional settings. Currently the only such setting is `show_for_owner`, which is false by default and must be set to yes to actually make it appear.

The map marker widget will also contain a circle around it denoting the progress of this action. This value can be tracked via the `progress` scripted value which is also given interaction target scopes like others. This value can also later be fetched using `Cabinet.GetActionNumProgress` Data function.

The action can be made to automatically cease if its `is_finished` trigger is fulfilled. The trigger will take in interaction targets.

Localisation

Cabinet actions have several required keys to localize:

- `<key>` to represent the name of the cabinet action. E.g "Generate Action"
- `<key>_desc` to represent the description of the cabinet action.
- `<key>_active` to represent the action of doing the cabinet action. E.g "Generating Actions" Visible on the map marker and serves as name for when the action is being used in the cabinet
- `<key>_action` to represent the name of the action. E.g "Generate Actions in [target_location.GetName]" Used in the final tooltip for selecting target.
- `<key>_action_progress`
- `<key>_action_progress_wordier`
- `<key>_action_progress_tooltip` to contain tooltip data. Used in the tooltip for the map marker as well as in the cabinet to track progress.

Other

There are several other game objects that utilize action syntax due to its high versatility.

Resolutions

Main article: [International Organization modding](#)

Resolutions also utilize the Interaction Target syntax for much more customization.

Missions

Main article: [Mission modding](#)

Missions utilize the Interaction Target syntax, which allows for great versatility with what can be targetted with mission tasks. The interactions are done when completing the mission, or, if the task is timed, before the start of the mission completion timer. Therefore, the scopes from interaction targets are available in `on_start`, `on_completion` and in `on_monthly`. AI will then evaluate the best possible targets for the mission when necessary.

Peace treaties

Main article: [Peace treaty modding](#)

Scripted Peace treaties utilize Interaction Target syntax to generate multiple peace deals based on the targets provided. If a scripted peace treaty is looking for a location to be affected in the target country, each match of the `select_trigger` will count for a peace treaty entry.

References



Scripted types	Advances • Art • Buildings • Bureaucracies • Casus belli • Characters • Concepts • Countries • Culture • Diplomacy • Diseases • Estates • Goods • Institutions • International organizations • Laws • Movements • Peace treaties • Pops • Religion • Subject types • Traits • Units • Wargoals
Map	Map • Map modes • Terrain
Graphics	3D Models • Interface • Graphical assets • Fonts • Flags
Audio	Music • Sound
Other	AI • Console commands • Checksum • Mods • Mod compatibility • Mod structure • Troubleshooting
Guides	Interface modding guide • Mod translation • Save-game editing • Settlement position modding guide
Tools	Arcanum • PDX DeepL • PDX Workshop Manager • Community Mod Toolkit • Add Your Tool to the Wiki

"https://eu5.paradoxwikis.com/index.php?title=Action_modding&oldid=30363" sayfasından alınmıştır

Sayfa en son 02.36, 20 Şubat 2026 tarihinde değiştirildi.

Aksi belirtilmedikçe içeriğin kullanımı Attribution-ShareAlike 3.0 lisansı kapsamında uygundur.

GAMES

- Discover
- Our Brands
- Browse
- Play on Paradox technology

COMMUNITY

- Paradox Forums
- Paradox Wikis
- Support

ABOUT

- News
- About us
- Careers
- Join our playtests
- Media contact

SOCIAL MEDIA

- Terms & Policies
- Legal Information
- EU Online Dispute Resolution
- Report Illegal Content
- Frequently Asked Questions
- Paradox Interactive corporate website

